System Verification and Validation Plan for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

October 30, 2026

Revision History

Date	Version	Notes
Date 1	1.0	Notes
Date 2	1.1	Notes

[The intention of the VnV plan is to increase confidence in the software. However, this does not mean listing every verification and validation technique that has ever been devised. The VnV plan should also be a **feasible** plan. Execution of the plan should be possible with the time and team available. If the full plan cannot be completed during the time available, it can either be modified to "fake it", or a better solution is to add a section describing what work has been completed and what work is still planned for the future. —SS]

[The VnV plan is typically started after the requirements stage, but before the design stage. This means that the sections related to unit testing cannot initially be completed. The sections will be filled in after the design stage is complete. the final version of the VnV plan should have all sections filled in. —SS]

Contents

1 Symbols, Abbreviations, and Acronyms **iv**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 1
- 2.3 Custom Documents . . . 1
- 2.4 Relevant Documentation . . . 2

3 Plan **2**
- 3.1 Verification and Validation Team . . . 2
- 3.2 SRS Verification . . . 2
- 3.3 Design Verification . . . 2
- 3.4 Verification and Validation Plan Verification . . . 3
- 3.5 Implementation Verification . . . 3
- 3.6 Automated Testing and Verification Tools . . . 3
- 3.7 Software Validation . . . 5

4 System Tests **6**
- 4.1 Tests for Functional Requirements . . . 6
  - 4.1.1 Area of Testing1 . . . 6
  - 4.1.2 Area of Testing2 . . . 7
- 4.2 Tests for Nonfunctional Requirements . . . 7
  - 4.2.1 Performance . . . 8
  - 4.2.2 Safety-Critical Requirements . . . 13
- 4.3 Traceability Between Test Cases and Requirements . . . 26

5 Unit Test Description **26**
- 5.1 Unit Testing Scope . . . 27
- 5.2 Tests for Functional Requirements . . . 27
  - 5.2.1 Module 1 . . . 27
  - 5.2.2 Module 2 . . . 28
- 5.3 Tests for Nonfunctional Requirements . . . 28
  - 5.3.1 Module ? . . . 28
  - 5.3.2 Module ? . . . 29
- 5.4 Traceability Between Test Cases and Modules . . . 29

6 Appendix **30**
6.1 Symbolic Parameters 30
6.2 Usability Survey Questions? 30

7 Appendix — Generative AI Use Disclosure **31**
7.1 AI Tools Used 31
7.2 How and Where Used 31
7.3 Verification of AI-Generated Content 31

8 Appendix — Reflection **32**

List of Tables

1 Automated testing and verification tools 3
[Remove this section if it isn't needed —SS]

List of Figures

[Remove this section if it isn't needed —SS]

1 Symbols, Abbreviations, and Acronyms

symbol	description
T	Test

[symbols, abbreviations, or acronyms — you can simply reference the SRS (Author, 2019) tables, if appropriate —SS]

[Remove this section if it isn't needed —SS]

This document ... [provide an introductory blurb and roadmap of the Verification and Validation plan —SS]

2 General Information

2.1 Summary

[Say what software is being tested. Give its name and a brief overview of its general functions. —SS]

2.2 Objectives

[State what is intended to be accomplished. The objective will be around the qualities that are most important for your project. You might have something like: "build confidence in the software correctness," "demonstrate adequate usability." etc. You won't list all of the qualities, just those that are most important. —SS]

[You should also list the objectives that are out of scope. You don't have the resources to do everything, so what will you be leaving out. For instance, if you are not going to verify the quality of usability, state this. It is also worthwhile to justify why the objectives are left out. —SS]

[The objectives are important because they highlight that you are aware of limitations in your resources for verification and validation. You can't do everything, so what are you going to prioritize? As an example, if your system depends on an external library, you can explicitly state that you will assume that external library has already been verified by its implementation team. —SS]

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

[Reference relevant documentation. This will definitely include your SRS and your other project documents (design documents, like MG, MIS, etc). You can include these even before they are written, since by the time the project is done, they will be written. You can create BibTeX entries for your documents and within those entries include a hyperlink to the documents. —SS]

Author (2019)

[Don't just list the other documents. You should explain why they are relevant and how they relate to your VnV efforts. —SS]

3 Plan

[Introduce this section. You can provide a roadmap of the sections to come. —SS]

3.1 Verification and Validation Team

[Your teammates. Maybe your supervisor. You should do more than list names. You should say what each person's role is for the project's verification. A table is a good way to summarize this information. —SS]

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

[Plans for design verification —SS]

[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.4 Verification and Validation Plan Verification

[The verification and validation plan is an artifact that should also be verified. Techniques for this include review and mutation testing. —SS]
[The review will include reviews by your classmates —SS]
[Create a checklists? —SS]

3.5 Implementation Verification

[You should at least point to the tests listed in this document and the unit testing plan. —SS]

[In this section you would also give any details of any plans for static verification of the implementation. Potential techniques include code walk-throughs, code inspection, static analyzers, etc. —SS]

[The final class presentation in CAS 741 could be used as a code walk-through. There is also a possibility of using the final presentation (in CAS741) for a partial usability survey. —SS]

3.6 Automated Testing and Verification Tools

This section names the tools that check the code automatically. They follow the expected technology and coding standard in the Development Plan. They are planned, not yet installed; the DevOps Lead (Shamil Canbolat) sets them up when the application scaffold exists, and the Testing Lead (Muhammad Huzaifah) chooses which tests run where. Table 1 lists each tool, what it checks and when it runs.

Table 1: Automated testing and verification tools

Purpose	**Tool**	**How and when it is used**
Language	TypeScript, strict mode	Next.js front end and server code. `tsc --noEmit` runs locally and on every PR; any type error fails the check.

Purpose	Tool	How and when it is used
Coding standard and linter	ESLint with the Next.js Core Web Vitals and TypeScript ESLint configurations	The coding standard in the Development Plan. Runs on every PR; any error fails the check. Used as evidence for NFR-MAINT-004.
Formatter	Prettier (check mode)	Runs on every PR; any unformatted file fails the check (NFR-MAINT-004).
Unit and component tests	Vitest with React Testing Library	Unit tests for logic such as stroke evaluation and save handling, and component tests for screens. Runs locally before review and on every PR; any failing test fails the check.
Code coverage	`@vitest/coverage-v8`	Produces a statement and branch coverage report on every PR (see below).
End-to-end tests	Playwright, on Chromium, WebKit and Firefox	Complete user flows, including the automated tests in the nonfunctional system tests (for example network loss and repeated failed attempts). Runs on every PR once a flow exists. Emulation does not replace physical stylus testing, so stylus tests are also run by hand on the test iPad.
Continuous integration	GitHub Actions	Runs all checks above on PRs to `development` and `main` and after each merge. A PR cannot be merged while a required check fails. The existing `latex-pages.yml` workflow continues to build the documents.

Purpose	Tool	How and when it is used
Performance measurement	Safari Web Inspector (Timelines), Chrome DevTools (Performance panel) and the browser Performance API (`performance.now()`)	Measures stroke latency, refinement time and page load for the performance tests. Safari Web Inspector, run on a Mac, reads timings from the test iPad; Chrome DevTools is used for runs on a laptop. Timings logged by Playwright scripts make the measurements repeatable.
Network inspection	Chrome DevTools (Network panel)	Records requests for the HTTPS and third-party data tests (NFR-OPS-001, SAF-SHA-002).

Code coverage. The coverage report is attached to every PR as a GitHub Actions artifact, and the change in coverage is noted in the PR review. The team does not set a fixed coverage target, because high coverage alone does not show that tests are good (Development Plan). Instead, reviewers check that new or changed logic has tests, and that modules tied to a safety-critical requirement (the safety-critical tests below), such as attempt saving, stroke evaluation and access checks, have every branch covered. Full branch coverage of these modules is a goal, not a guarantee: some branches (for example, a server failure in the middle of a request) may be hard to reach in a test. The actual coverage is reported honestly in the VnV Report as a summary table of statement and branch coverage per module, and any safety-critical branch left uncovered is listed with the reason and the test or inspection that covers it instead.

Limits of the tools. Passing these checks shows that the code builds, follows the coding standard and passes its tests. It does not show that the tests are complete, so the code inspections planned under Implementation Verification remain necessary.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that

here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

[There should be text between all headings, even if it is just a roadmap of the contents of the subsections. —SS]

4.1 Tests for Functional Requirements

[Subsets of the tests may be in related, so this section is divided into different areas. If there are no identifiable subsets for the tests, this level of document structure can be removed. —SS]

[Include a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. —SS]

4.1.1 Area of Testing1

[It would be nice to have a blurb here to explain why the subsections below cover the requirements. References to the SRS would be good here. If a section covers tests for input constraints, you should reference the data constraints table in the SRS. —SS]

Title for Test

1. test-id1

Control: Manual versus Automatic

Initial State:

Input:

Output: [The expected result for the given inputs. Output is not how you are going to return the results of the test. The output is the expected result. —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Control: Manual versus Automatic

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

4.1.2 Area of Testing2

...

4.2 Tests for Nonfunctional Requirements

[The nonfunctional requirements for accuracy will likely just reference the appropriate functional tests from above. The test cases should mention reporting the relative error for these tests. Not all projects will necessarily have nonfunctional requirements related to accuracy. —SS]

[For some nonfunctional tests, you won't be setting a target threshold for passing the test, but rather describing the experiment you will do to measure the quality for different inputs. For instance, you could measure speed versus the problem size. The output of the test isn't pass/fail, but rather a summary table or graph. —SS]

[Tests related to usability could include conducting a usability test and survey. The survey will be in the Appendix. —SS]

[Static tests, review, inspections, and walkthroughs, will not follow the format for the tests given below. —SS]

[If you introduce static tests in your plan, you need to provide details. How will they be done? In cases like code (or document) walkthroughs, who will be involved? Be specific. —SS]

4.2.1 Performance

These tests cover the performance requirements in SRS Section 12 (NFR-PERF-001 to NFR-PERF-005, NFR-PERF-008 and NFR-PERF-009). Timing thresholds are written as symbolic constants. The SRS leaves their values to this plan, so each constant in the Symbolic Parameters appendix states how its value is set and when.

Test environment. Squigglr is a web app, so in principle it runs on any device with a modern browser, and on any network. Testing every device and every connection is not realistic for a team of five, and it is not needed to show that the requirements are met. The team will instead run the timing tests on one tablet, an iPad with a stylus, which is the device the app is designed around (SRS ASM-003). The tests run in a controlled setting: the iPad is charged, Low Power Mode is off, every other app and browser tab is closed, automatic updates are paused, and the iPad uses the same Wi-Fi network for every run. Keeping these conditions the same means that a change in a measured time comes from a change in Squigglr, not from the device or the network. Automatic tests that do not need a physical stylus run in Playwright on a team laptop under the same rules.

Why each test is repeated. Even in a controlled setting, one measurement can be slow by chance, for example when the browser collects garbage. Each timing test is therefore repeated `PERF_RUNS` times, and the VnV Report gives the minimum, median, 95th percentile and maximum of the measured times next to the threshold. A test passes only when the 95th percentile and the maximum are both within the threshold. This way a single slow run is still visible, but one unlucky run does not decide the result.

Limits. Because the tests use one device and one network, they show that Squigglr meets its timing requirements on the device it is designed for. They do not show how it performs on older devices or slower networks. Other browsers and devices are checked for correct operation, not speed, in T-PERF-007.

1. T-PERF-001: Stroke feedback latency

   Purpose: a child should see each stroke appear as they draw it; a visible delay makes drawing feel broken and can frustrate a child who is still building fine motor control.

   Type: Dynamic, automatic (Playwright) and manual (physical stylus).

   Initial State: a guided lesson is open at each stage type (demonstration, tracing, reduced guidance and independent drawing) with no earlier attempts; no other tabs are open.

   Input: the prepared stroke files `line.json`, `circle.json` and `square-4-strokes.json`, replayed as pointer events by Playwright, `PERF_RUNS` times each; then `PERF_RUNS` strokes drawn by hand with the stylus on the iPad.

   Output: for every stroke, the time from the last pointer event to the first frame that shows the stroke is at most `FEEDBACK_LATENCY_MAX`.

   Test Case Derivation: NFR-PERF-001 requires visual acknowledgement within `FEEDBACK_LATENCY_MAX` of the final input point. Multi-stroke shapes are included because they create the most rendering work per attempt. The replayed strokes give exactly the same input on every run, so they are the main measurement. A person cannot draw the same stroke twice, so the hand-drawn strokes are not meant to be identical: only the time to show each stroke is measured, not how accurate it is. They are there because replayed events skip the real stylus input path on the iPad, which could add its own delay.

   How test will be performed: the replay script stamps `performance.now()` at the last pointer event and in the next `requestAnimationFrame` callback after the stroke is drawn. For the manual runs, the iPad is connected to a Mac and the same interval is read from a Safari Web Inspector timeline recording. Times are summarized as described above.

2. T-PERF-002: Refinement latency and canvas responsiveness

   Purpose: refinement runs on an outside service that can be slow, so this checks that the refined drawing arrives in time and that the child can keep drawing while it waits.

   Type: Dynamic, automatic.

   Initial State: the creative space is open with refinement on and the refinement service reachable.

   Input: the prepared drawings `house.json` and `sun.json` are replayed and finished, `PERF_RUNS` times each; during the wait, a further stroke is replayed every 200 ms.

   Output: the refined result appears within `REFINE_LATENCY_MAX` of finishing, and every stroke replayed during the wait is drawn and saved without an error.

   Test Case Derivation: NFR-PERF-002 requires both the time limit and an unblocked canvas while the refinement is pending.

   How test will be performed: the script records the time from the finish action to the first frame showing the refined result, and checks the canvas contents after each extra stroke.

3. T-PERF-003: Progress summary load time

   Purpose: a parent or educator with a long history of saved attempts should not be left waiting for the progress summary to load.

   Type: Dynamic, automatic.

   Initial State: an adult account with one child profile whose history holds `HISTORY_SIZE_LARGE` attempts across three lessons, logged in, with the browser cache cleared.

   Input: the adult opens the progress summary ten consecutive times.

   Output: each time, the summary is fully visible within `SUMMARY_LOAD_MAX`.

   Test Case Derivation: the fit criterion of NFR-PERF-003 names ten consecutive requests. A large history is used because load time grows with the number of records.

How test will be performed: Playwright measures from the click to the moment the last chart element is visible. The test is repeated with histories of 10 attempts and `HISTORY_SIZE_LARGE` attempts, and the two sets of times are reported together to show how load time grows with history size.

4. T-PERF-004: Deterministic stage result

   Purpose: the adult relies on pass and fail results to choose the next level, so the same drawing must always get the same result.

   Type: Dynamic, automatic (unit test in Vitest).

   Initial State: the stage evaluation module is loaded with the criteria for each prepared stage.

   Input: the prepared stroke files from T-PERF-001, plus `circle-near-miss.json`, a drawing just outside `STAGE_PASS`, each evaluated ten consecutive times.

   Output: all ten results for each input are identical, including the pass or fail decision.

   Test Case Derivation: NFR-PERF-004 requires the same strokes and criteria to give the same result. The near miss checks that a borderline result does not change between runs.

   How test will be performed: a Vitest test calls the evaluation function directly and compares every result with the first. A second run in Playwright replays the same strokes through the user interface, to confirm that input handling does not change the result.

5. T-PERF-005: Submitting attempts after reconnection

   Purpose: home and school Wi-Fi can drop; a child's work done while offline must still reach the server once the connection returns, without being lost or saved twice.

   Type: Dynamic, automatic.

   Initial State: a guided lesson is open; the network is online.

   Input: the network is set offline in Playwright; the prepared attempt `circle.json` is completed; the network is restored. This is done three

consecutive times, then repeated with three attempts completed while offline.

Output: after each reconnection, every attempt completed offline is saved on the server exactly once, in the order made, and none is lost or duplicated.

Test Case Derivation: NFR-PERF-005 names three consecutive runs. The three-attempt case checks the order of a queue and catches duplicates.

How test will be performed: after each run, the script reads the child's attempt records through the test database and compares them with the attempts made. This test is shared with SAF-RAY-003 and SAF-SHA-003 (safety-critical tests below).

6. T-PERF-006: Concurrent session capacity

   Purpose: several classrooms or households may use Squigglr at once; this checks that the speed limits still hold under that load.

   Type: Dynamic, automatic (load test).

   Initial State: the preview deployment, with `CONCURRENT_CAPACITY` test adult accounts and one child profile each.

   Input: a load script runs `CONCURRENT_CAPACITY` simulated sessions at once; each replays a lesson of ten attempts and then opens the progress summary.

   Output: no request fails, and the measurements of T-PERF-001 to T-PERF-003, taken in one extra session during the load, stay within their thresholds.

   Test Case Derivation: NFR-PERF-008 requires the speed limits to hold under the agreed load.

   How test will be performed: the load script runs simulated sessions in Playwright; one further session takes the timing measurements. The run is repeated at a quarter, half and all of `CONCURRENT_CAPACITY`, and the times are plotted against the number of sessions in the VnV Report.

7. T-PERF-007: Squigglr keeps working until the end of the capstone

   Purpose: squigglr must keep working on the supported devices and browsers at least until the end of the capstone, the final demo on 15 March 2027 (PH-3 in the SRS). Browsers update often, and an update can break a feature without any change to our code, so this checks every feature at each milestone.

   Type: Dynamic, manual.

   Initial State: the release build at each milestone: the proof-of-concept demo (PH-1, 23 November 2026), Rev 0 (PH-2, 1 February 2027) and the final demo (PH-3, 15 March 2027).

   Input: on the iPad and on each other combination in `SUPPORTED_DEVICES` that the team has access to, a team member follows the checklist of every feature: create an account and a child profile, run one guided lesson to the end, draw in the creative space, restore an original, and open the progress summary.

   Output: every checklist item works on every combination. Any combination that fails is removed from `SUPPORTED_DEVICES`, and OI-09 is updated.

   Test Case Derivation: the fit criterion of NFR-PERF-009.

   How test will be performed: a team member runs the checklist by hand at each milestone. Running it at the two earlier milestones, not only at the final demo, means a browser update that breaks a feature is found while there is still time to fix it. The results are kept as a table of device by feature in the VnV Report.

4.2.2 Safety-Critical Requirements

These tests cover the 18 safety-critical requirements in SRS Section 12.2, which come from the Hazard Analysis. A failure here could harm a child's well-being, privacy or records, so each requirement gets two different kinds of evidence: a test below, and the safety inspection described next. Every test below is dynamic except T-SAF-004, which checks wording in templates and so is an inspection; T-SAF-016 combines a code review with recordings on the iPad. The tests try to break each safeguard, for example by having a child tap adult-only controls, cutting the connection during an attempt, or

switching between children on a shared device. All tests use test accounts and prepared data only; no child takes part (SRS CON-005). In these tests, "the adult" means the parent or educator responsible for the child, and "the adult check" means the check in `ADULT_CHECK` (for example a PIN or password) that only the adult can complete.

Safety inspection (SI). Before each revision (Rev 0 and Rev 1), the code that implements each requirement below is inspected by two team members who did not write it. The Backend Lead (Rayan Nasrallah) runs the inspection and the Testing Lead (Muhammad Huzaifah) records the results. For each requirement, the inspectors:

1. find every code path that triggers the safeguard, using the test-to-requirement traceability table;
2. check that no path skips it, for example an adult-only action reachable from a URL or a save that reports success before the server responds;
3. check that a dynamic test exercises each path, and add one if not;
4. check the Code checklist items on error handling and input validation;
5. record each finding as a GitHub issue labelled `safety`, and re-inspect after the fix.

The inspection passes when every requirement has no open `safety` issue. Results are reported in the VnV Report.

Why this is enough. Testing cannot prove that a safeguard always works; it can only raise confidence that it does. One test per requirement, each with cases chosen to break the safeguard, plus an inspection that looks for paths the test missed, gives two independent checks. A second dynamic test is added wherever the inspection finds an untested path. The inspection covers only the code tied to these 18 requirements, because a full review of every module by two people is not realistic in the time the team has. The rest of the code is covered by the code checklist and the normal PR review.

How a lesson works. Several of these tests check the limits in a guided lesson, so this paragraph summarizes the lesson flow from SRS BR-002, UC-002 to UC-005 and the state machine in SRS Section 9.2. The adult picks the child and the lesson and completes the adult check. The child then watches a demonstration and works through three stages in order: tracing a full outline, drawing between endpoints, and drawing with no guide. An attempt that meets `STAGE_PASS` moves the child to the next stage automatically, and a pass at the last stage completes the lesson. Failed attempts are counted *per stage*, and the count starts again whenever the stage changes:

- after `DIFFICULTY_STREAK` failed attempts in a row, Squigglr suggests to the adult that the child go back a stage; the stage changes only if the adult completes the adult check and accepts, and the child keeps practising while the suggestion waits;
- after `ATTEMPT_LIMIT` failed attempts at one stage, the lesson ends automatically, unless the child is improving, in which case the child gets up to `ATTEMPT_EXTENSION` more attempts, never more than `ATTEMPT_LIMIT_MAX` in total at that stage;
- after `LESSON_IDLE_PROMPT` with no input, Squigglr shows the child a check-in, and after `LESSON_IDLE_END` with no input, the lesson ends; time without input never counts as a failed attempt.

Because `ATTEMPT_LIMIT` is always larger than `DIFFICULTY_STREAK` (SAF-RAY-008), the adult always receives the suggestion before the lesson ends: the suggestion is the early warning and the limit is the backstop. The adult sets these values for each child within ranges set by the team. The examples below use illustrative values (`DIFFICULTY_STREAK` = 3, `ATTEMPT_LIMIT` = 6, `ATTEMPT_EXTENSION` = 2, `ATTEMPT_LIMIT_MAX` = 8) to show the expected behaviour; the tests themselves use the ends of each range.

1. T-SAF-001: Adult-only actions need the adult check

   Purpose: choosing a lesson, choosing a starting stage, switching child and accepting a go-back suggestion belong to the adult, so a child must not be able to complete any of them.

   Type: Dynamic, manual and automatic. Covers SAF-RAY-001.

Initial State: an adult account with two child profiles; child A is in a guided lesson, and a go-back suggestion is waiting for the adult.

Input: on the lesson screen, the tester (acting as the child) taps every visible control, taps the screen 50 times at random positions, opens the web address of each adult-only action directly, uses the browser's back and forward buttons and reloads the page. Each time the adult check appears, the tester leaves it or enters a wrong answer. Finally, the tester (acting as the adult) completes the adult check and accepts the suggestion.

Output: every adult-only action asks for the adult check, and none completes until the check is passed. The suggestion stays waiting and the child can keep drawing at the current stage. After the adult passes the check and accepts, the child moves back one stage.

Example: child A fails three attempts in a row at the endpoints stage, and a go-back suggestion appears. Child A taps "accept"; Squigglr asks for the adult check, which child A cannot complete, so the child stays at the endpoints stage and keeps practising. The adult then completes the check and accepts, and child A moves back to the tracing stage.

Test Case Derivation: SAF-RAY-001 and its hazards H-RAY-001 and H-RAY-006. Random taps imitate a child who cannot read exploring the screen.

How test will be performed: a Playwright script performs the taps, web addresses and navigation and checks the lesson state after each one; a tester repeats the taps by hand with the stylus on the iPad.

2. T-SAF-002: Notice on an unsupported device, browser or input

   Purpose: drawing results may be unreliable on a device or browser that is not in `SUPPORTED_DEVICES`, or when the child draws with a finger instead of a stylus, so the adult must be told and can decide whether to continue.

   Type: Dynamic, automatic and manual. Covers SAF-RAY-002.

   Initial State: the adult has chosen a lesson; it has not started.

   Input: (a) the lesson is started once with each combination in `SUPPORTED_DEVICES`, and once with each of three combinations that are not in it: an older

browser version, a phone-sized screen, and a browser without pointer event support; (b) on the iPad, a supported lesson is started and the first stroke is drawn with a finger; (c) the same, with the first stroke drawn with the stylus.

Output: (a) for each unsupported combination, a notice the adult can read appears before the first attempt; for each supported combination, no notice appears; (b) the notice appears after the first stroke; (c) no notice appears.

Test Case Derivation: the fit criterion of SAF-RAY-002 (H-RAY-002). A browser cannot tell whether a stylus is connected before it is used, so case (a) checks only what the browser reports (version, screen size and pointer event support), and cases (b) and (c) check the input type the browser reports for the first stroke (pen or touch).

How test will be performed: for (a), Playwright sets the browser version, screen size and pointer event support for each combination and checks for the notice; for (b) and (c), a tester draws on the iPad, once with a finger and once with the stylus.

3. T-SAF-003: Attempts kept until the server confirms them

   Purpose: an attempt must never look saved when it is not, or the adult would make decisions from incomplete records.

   Type: Dynamic, automatic. Covers SAF-RAY-003 and SAF-SHA-003.

   Initial State: a guided lesson is open and online.

   Input: (a) the connection is cut during an attempt; (b) the connection is cut after the attempt is finished but before the server confirms the save, and the page is then reloaded, and separately closed and reopened, while offline; (c) the server returns an error for `FAILED_SAVES` saves in a row; (d) `FAILED_SAVES` saves succeed normally.

   Output: in (a) to (c), no attempt is shown as saved before the server confirms it, the adult is told which attempts are not saved, and every attempt is saved once the connection or server is back; in (d), no failure message appears.

   Test Case Derivation: SAF-RAY-003 and SAF-SHA-003 (H-RAY-002, H-SHA-003). Case (d) checks that the warning does not appear when

nothing failed.

How test will be performed: Playwright turns the network off and makes the server return errors, then compares the server's records with the attempts made.

4. T-SAF-004: No diagnosis or comparison in summaries and exports

   Purpose: Squigglr is a practice tool, not an assessment, so no summary or export may label, diagnose or compare a child, and each must say so.

   Type: Static (inspection), manual. Covers SAF-RAY-004.

   Initial State: the templates for every summary and export at that revision.

   Input: one team member who did not write the templates reads each one.

   Output: every template states that it describes practice only and is not an assessment or diagnosis, and none uses a diagnostic label, judging wording (for example "struggling", "behind" or "below average") or a comparison with other children.

   Test Case Derivation: the fit criterion of SAF-RAY-004 (H-RAY-003), which asks for an inspection of every summary and export template. The team does not intend to add diagnoses, so the realistic risks are a new template that leaves out the required statement and wording that judges the child; a read of each template catches both.

   How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

5. T-SAF-005: Lesson ends automatically at the attempt limit

   Purpose: a child must not be left repeating a stage they keep failing, so the lesson must end by itself at the limit the adult set.

   Type: Dynamic, automatic. Covers SAF-RAY-005.

Initial State: a child profile with `ATTEMPT_LIMIT` and `ATTEMPT_LIMIT_MAX` at the lowest values in their ranges; the test is then repeated with the highest values.

Input: (a) at the tracing stage, the failing attempt `circle-fail.json` is replayed until the lesson ends; (b) the child passes tracing after a few failures, and the failing attempt is replayed at the endpoints stage until the lesson ends; (c) a sequence of attempts that improves enough to earn an extension is replayed until the lesson ends.

Output: the lesson ends after exactly `ATTEMPT_LIMIT` failed attempts at one stage, or `ATTEMPT_LIMIT_MAX` when extended, with no action from the child; failures at an earlier stage do not count at the new stage; and the saved record matches the attempts made, including the furthest stage reached.

Example: the sixth failed tracing attempt ends the lesson. In case (b), a child who passed tracing after four failures still gets six attempts at the endpoints stage. In case (c), an improving child gets attempts 7 and 8, and the lesson ends after the eighth failure.

Test Case Derivation: SAF-RAY-005 (H-RAY-004). Testing both ends of the ranges follows the supervisor's advice to test at the minimum and maximum (SRC-008).

How test will be performed: Playwright counts the attempts and checks that no attempt is accepted after the lesson ends.

6. T-SAF-006: Limit endings are shown to the adult across repeated lessons

   Purpose: each time a lesson is done, it is a new lesson record, so the adult must see how often the lesson ended at the limit across all of them; doing the lesson again must not hide earlier failures.

   Type: Dynamic, automatic. Covers SAF-RAY-006.

   Initial State: as in T-SAF-005.

   Input: the adult starts the lesson and it is run until it ends at the limit; the adult then starts the same lesson again, and this is repeated so the lesson is done three times. In the third run, an improving sequence earns an extension before the lesson ends at the limit.

Output: the adult's view of that lesson shows three lesson records, each ending at the limit, one extension, and every attempt from all three records.

Example: the adult sees "ended at the limit: 3 times; extended: once" and all $6 + 6 + 8 = 20$ attempts.

Test Case Derivation: SAF-RAY-006 (H-RAY-005).

How test will be performed: the script compares the counts and the list of attempts in the adult's view with the runs made.

7. T-SAF-007: Attempts saved only to the active child

   Purpose: on a shared device, one child's attempts must never be saved to another child's record.

   Type: Dynamic, automatic and manual. Covers SAF-RAY-007.

   Initial State: one adult account with three child profiles (A, B and C) on one device.

   Input: the adult switches between the children in the order A, B, A, C, and each child makes two attempts; the connection is cut once during child B's turn.

   Output: every practice screen shows the active child's avatar, and every attempt and suggestion is saved under the child whose avatar was shown when it was made.

   Test Case Derivation: SAF-RAY-007 (H-RAY-006). The lost connection checks that a delayed save still goes to the right child.

   How test will be performed: Playwright checks each child's records; a tester checks by hand that each avatar can be recognized without reading.

8. T-SAF-008: Adult-set values stay within their ranges

   Purpose: a mistyped or extreme value could give a child an impossible lesson, so each value must stay within the range the team set and only the adult may change it.

   Type: Dynamic, automatic. Covers SAF-RAY-008.

Initial State: a new child profile with no values chosen.

Input: (a) no value is set; (b) each value is set to its minimum, its maximum, one below the minimum and one above the maximum; (c) `ATTEMPT_LIMIT` is set equal to `DIFFICULTY_STREAK`, and `ATTEMPT_LIMIT_MAX` is set one below `ATTEMPT_LIMIT`; (d) a change is tried without completing the adult check.

Output: (a) each value is the middle of its range; (b) in-range values are saved and out-of-range values are refused; (c) both settings are refused; (d) nothing changes. The values in use are shown with the child's records.

Example: if the team-set range for `ATTEMPT_LIMIT` were 4 to 10, an unset value would be 7, values 4 and 10 would be saved, and 3 and 11 would be refused. With `DIFFICULTY_STREAK` = 3, an `ATTEMPT_LIMIT` of 3 is refused, because the lesson would end before the adult received the go-back suggestion.

Test Case Derivation: the fit criterion of SAF-RAY-008 (H-RAY-007), using boundary values on each side of each range.

How test will be performed: a Vitest test checks the validation function with every case, and Playwright repeats cases (b) to (d) through the settings screen and through direct server requests.

9. T-SAF-009: Duplicate input events counted once

   Purpose: a stylus event reported twice must not change a drawing's result, or a child could pass or fail for no real reason.

   Type: Dynamic, automatic. Covers SAF-AHM-001.

   Initial State: a guided lesson at the tracing stage.

   Input: `square-4-strokes.json` is replayed as recorded, then with every input event sent twice; then `overlap-2-strokes.json`, two separate strokes that cross, is replayed.

   Output: the recorded points, classification and pass result are the same with and without the duplicates, and both crossing strokes are kept.

   Test Case Derivation: the fit criterion of SAF-AHM-001 (H-AHM-002).

   How test will be performed: Playwright replays both versions and compares the saved records.

10. T-SAF-010: Drawing kept when the canvas changes size

    Purpose: rotating the iPad or resizing the window must not distort a drawing or change its result.

    Type: Dynamic, automatic. Covers SAF-AHM-002.

    Initial State: a guided lesson with an attempt partly drawn.

    Input: the window is resized and the iPad is rotated during the attempt; then input events with positions outside the canvas, missing positions (`NaN`) and very large values are sent.

    Output: the order and relative positions of the points are unchanged, and the classification is the same as without resizing; none of the invalid points is counted or makes the attempt pass.

    Test Case Derivation: the fit criterion of SAF-AHM-002 (H-AHM-003).

    How test will be performed: Playwright resizes the drawing area and compares the scaled points with the original; a Vitest test sends the invalid values to the input handler.

11. T-SAF-011: Every data request checks the account

    Purpose: one family's or school's data must never be reachable from another account.

    Type: Dynamic, automatic. Covers SAF-SHA-001, with T-SEC-001.

    Initial State: two adult accounts, A and B, each with a child profile and saved attempts.

    Input: logged in as A, the tester sends requests to read, change and delete B's child profiles, attempts, settings and summaries to every server endpoint, then repeats them with no session and with an expired session.

    Output: every request is refused, and none of B's data is returned or changed.

    Test Case Derivation: the fit criterion of SAF-SHA-001 (H-SHA-001).

    How test will be performed: a script lists every endpoint from the code and sends each request; the list is checked in the safety inspection, so a new endpoint cannot be missed.

12. T-SAF-012: No personal information sent to outside services

    Purpose: children's and adults' personal information must not leak to outside services, including the refinement service.

    Type: Dynamic, manual. Covers SAF-SHA-002.

    Initial State: an account whose adult name and email, and the child profile's identifier, are unique marker strings.

    Input: the tester uses every feature, including refinement, while recording all network traffic.

    Output: no request to an outside service contains any marker string, and requests go only to approved destinations.

    Test Case Derivation: the fit criterion of SAF-SHA-002 (H-SHA-002). Unique markers make any leak easy to find.

    How test will be performed: the test runs in Chrome on a laptop; the DevTools Network panel is exported as a HAR file and searched for the markers and for destinations that are not approved.

13. T-SAF-013: Logout after inactivity

    Purpose: an adult who walks away from a shared device must not leave their account and their children's data open.

    Type: Dynamic, automatic. Covers SAF-SHA-006.

    Initial State: an adult is logged in.

    Input: (a) no activity for `IDLE_TIMEOUT`; (b) activity just before `IDLE_TIMEOUT`, then no activity.

    Output: the adult is logged out after `IDLE_TIMEOUT` of inactivity and not before, and any later request using that session is refused.

    Test Case Derivation: SAF-SHA-006 (H-SHA-001). Case (b) checks that activity restarts the timer.

    How test will be performed: Playwright uses a controlled clock to move time forward and checks the session.

14. T-SAF-014: Feedback does not draw the missing part

Purpose: feedback should show the child what is missing without drawing it for them, so the record shows only the child's own work.

Type: Dynamic, automatic. Covers SAF-MOU-001.

Initial State: a guided lesson at each predefined difficulty.

Input: `square-missing-side.json`, an attempt with one side missing.

Output: the feedback shows where the side is missing, and the strokes on screen and in the record are the same before and after the feedback.

Test Case Derivation: the fit criterion of SAF-MOU-001 (H-MOU-004).

How test will be performed: Playwright compares the stroke data before and after the feedback is shown.

15. T-SAF-015: Assisted, independent and creative work kept apart

    Purpose: the adult must be able to tell what the child drew with an on-screen guide from what they drew alone, so progress is not overstated.

    Type: Dynamic and static, manual. Covers SAF-MOU-002.

    Initial State: a prepared history of one lesson record holding 10 assisted attempts, 10 independent attempts, and 5 creative space drawings made separately.

    Input: every summary and record view is opened.

    Output: assisted and independent attempts are counted and shown separately, no creative space drawing appears, and no view ranks or rates the child by the number of attempts.

    Example: a lesson record with 10 successful attempts while tracing and 2 while drawing alone is summarized as "10 with a guide, 2 without", never as "12 successful drawings".

    Test Case Derivation: the fit criterion of SAF-MOU-002 (H-MOU-005). The assisted or independent label belongs to each attempt, not to the lesson: following FR-LRN-012, an attempt is assisted when it is drawn with an on-screen guide (the tracing or endpoints stage) and independent when it is drawn with no guide. One lesson record normally holds both kinds. Help from an adult outside Squigglr, such as guiding the child's hand, is not detected or recorded, and this test does not cover it.

How test will be performed: a tester checks each view against the prepared history, and a Vitest test of the summary function compares the counts.

16. T-SAF-016: No more than three flashes per second

    Purpose: rapid flashing can trigger seizures, so no child-facing screen may flash more than three times in any one second.

    Type: Static (code review) and dynamic, manual. Covers SAF-MOU-003.

    Initial State: the code for every animation and transition on child-facing screens, and Squigglr open on the iPad with a guided lesson ready.

    Input: (a) a team member who did not write it reviews every animation and transition used on child-facing screens, which are all defined in Squigglr's own code and style sheets; (b) on the iPad, the screens that animate (all feedback, the lesson-end message, the inactivity check-in and the screen transitions) are screen-recorded.

    Output: (a) no animation or transition changes brightness or colour more than three times in any one second; (b) stepping through each recording frame by frame, no one-second window has more than three flashes.

    Test Case Derivation: the fit criterion of SAF-MOU-003 and WCAG 2.2 success criterion 2.3.1 (World Wide Web Consortium, 2023). Because every animation comes from our own code, reviewing that code finds a flashing effect before it reaches a screen; the recordings then confirm the result on the device, without recording every screen.

    How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

17. T-SAF-017: Check-in and lesson ending after inactivity

    Purpose: a child may walk away during a lesson, and on a shared device another child could carry on under their profile, so an idle lesson must check in with the child and then end, with a clear record for the adult.

Type: Dynamic, automatic and manual. Covers SAF-RAY-009.

Initial State: a guided lesson at the endpoints stage, with `LESSON_IDLE_PROMPT` and `LESSON_IDLE_END` at the lowest values in their ranges; the test is then repeated with the highest values.

Input: (a) no input for `LESSON_IDLE_PROMPT`, then a touch; (b) no input for `LESSON_IDLE_END`; (c) two failed attempts, then no input until the lesson ends.

Output: (a) the check-in appears at `LESSON_IDLE_PROMPT`, the touch dismisses it, and the lesson continues at the same stage; (b) the check-in appears, then the lesson ends at `LESSON_IDLE_END`, the progress is saved, and the adult's view shows the lesson ended for inactivity; (c) the record shows exactly two failed attempts, so the idle time added none.

Example: with illustrative values of 60 seconds for the check-in and 120 seconds for the end, a child who stops drawing sees an animated character at 60 seconds; if nobody touches the screen, the lesson ends at 120 seconds and the adult sees "ended: no activity".

Test Case Derivation: the fit criterion of SAF-RAY-009 (H-RAY-006). Case (c) checks that inactivity is never counted as failure.

How test will be performed: Playwright uses a controlled clock to move time forward and checks the screen and the saved record; a tester checks by hand on the iPad that the check-in can be understood without reading and does not flash.

4.3 Traceability Between Test Cases and Requirements

[Provide a table that shows which test cases are supporting which requirements. —SS]

5 Unit Test Description

[This section should not be filled in until after the MIS (detailed design document) has been completed. —SS]

[Reference your MIS (detailed design document) and explain your overall philosophy for test case selection. —SS]

[To save space and time, it may be an option to provide less detail in this section. For the unit tests you can potentially layout your testing strategy here. That is, you can explain how tests will be selected for each module. For instance, your test building approach could be test cases for each access program, including one test for normal behaviour and as many tests as needed for edge cases. Rather than create the details of the input and output here, you could point to the unit testing code. For this to work, you code needs to be well-documented, with meaningful names for all of the tests. —SS]

5.1 Unit Testing Scope

[What modules are outside of the scope. If there are modules that are developed by someone else, then you would say here if you aren't planning on verifying them. There may also be modules that are part of your software, but have a lower priority for verification than others. If this is the case, explain your rationale for the ranking of module importance. —SS]

5.2 Tests for Functional Requirements

[Most of the verification will be through automated unit testing. If appropriate specific modules can be verified by a non-testing based technique. That can also be documented in this section. —SS]

5.2.1 Module 1

[Include a blurb here to explain why the subsections below cover the module. References to the MIS would be good. You will want tests from a black box perspective and from a white box perspective. Explain to the reader how the tests were selected. —SS]

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

Test Case Derivation: [Justify the expected value given in the Output field —SS]

How test will be performed:

2. test-id2

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

   Initial State:

   Input:

   Output: [The expected result for the given inputs —SS]

   Test Case Derivation: [Justify the expected value given in the Output field —SS]

   How test will be performed:

3. ...

5.2.2 Module 2

...

5.3 Tests for Nonfunctional Requirements

[If there is a module that needs to be independently assessed for performance, those test cases can go here. In some projects, planning for nonfunctional tests of units will not be that relevant. —SS]

[These tests may involve collecting performance data from previously mentioned functional tests. —SS]

5.3.1 Module ?

1. test-id1

   Type: [Functional, Dynamic, Manual, Automatic, Static etc. Most will be automatic —SS]

Initial State:

Input/Condition:

Output/Result:

How test will be performed:

2. test-id2

   Type: Functional, Dynamic, Manual, Static etc.

   Initial State:

   Input:

   Output:

   How test will be performed:

5.3.2 Module ?

...

5.4 Traceability Between Test Cases and Modules

[Provide evidence that all modules have been considered. —SS]

References

Author Author. System requirements specification. https://github.com/..., 2019.

World Wide Web Consortium. Web content accessibility guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/, 2023. W3C Recommendation.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

8 Appendix — Reflection

[This section is not required for CAS 741 —SS]

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Rayan: Something that really went well was talking to Chris, our TA, before my sections were finished. In our meeting on October 28, 2026, I asked him about the things I was unsure of: whether thresholds we cannot set yet can be decided by measurement, how many tests a safety requirement needs, and whether we need a fixed code coverage target. His answers helped me narrow down which testing strategies to use and how to run certain tests, and showed me what in my first draft was not clear yet, for example that every TBD value in the SRS should point to the method in this plan, and that I needed to justify why one test plus a safety inspection gives enough confidence.

   Our process also went better than last time. In the SRS and Hazard Analysis we had many merge conflicts, mostly because everyone added rows to the same revision history table. This time I suggested that everyone send their revision history entries to Moustafa, who is doing the final quality check, and he adds them all at the end. I also split my own part into five sub-issues, each on its own branch with its own pull request, so each part could be reviewed on its own, and when I tested merging all of them together they merged without conflicts.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Rayan: The main pain point was that writing the plan kept sending me back to the SRS. Every test has to match a requirement, so each time a meeting or a test changed how something should work, I had to go back through the SRS, change it, and then check that my tests still matched it. The main changes came from two meetings:

   - *Our supervisor meeting on October 28, 2026.* Dr. Anand advised that we should not collect children's personal details, so a child's profile now holds only a generated identifier and an avatar instead of a first name and username. This changed the business rules, the data dictionary, a use case, a security requirement and the Hazard Analysis, as well as my security test.
   - *Our TA meeting on the same day.* Chris advised that a value left as TBD should say how it will be decided, so every TBD performance value in the SRS now points to the measurement method in this plan.

   I resolved this by keeping all of these SRS changes in one issue and one pull request, separate from my VnV branches, and by re-reading each affected test against the updated requirement before marking a branch ready for review.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   Rayan: For my part of the plan (performance, safety-critical, security and compliance tests), I need three things I do not yet have:

   - *End-to-end test automation with Playwright*, in particular simulating network loss, replaying recorded stylus input and controlling the clock. Most of my safety tests depend on these, for example T-SAF-003, T-SAF-013 and T-SAF-017.

- *Security testing of a web back end*: checking access policies in the database, reusing session tokens and sending injection probes. As Backend Lead, I am responsible for the code these tests target.
- *Performance measurement on the iPad*: taking repeatable timings in the browser and summarizing them fairly, so that the thresholds the SRS leaves open can be set from evidence.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   Rayan:

   - *Playwright.* Two approaches: work through the official Playwright documentation and examples on network emulation and clocks, or learn from the first tests the Testing Lead writes. I will start with the documentation and write T-PERF-005 as a first test as soon as the scaffold exists. This is because the network and clock features are exactly what my tests need, and writing a real test early shows quickly what I am missing.
   - *Security testing.* Two approaches: follow the OWASP Web Security Testing Guide for the checks that apply to us, or rely on the hosting provider's security advice and default settings. I will use the OWASP guide, because it gives a checklist we can repeat each release, while default settings alone would not show us that our own access rules are correct.
   - *Performance measurement.* Two approaches: learn the browser developer tools (Safari Web Inspector for the iPad, Chrome DevTools on a laptop), or write timing code with the browser Performance API. I will do both, starting with the Performance API in Playwright scripts so the measurements can be repeated, and using Safari Web Inspector to understand any slow result on the iPad.